AI-ENABLED WEB PROJECT

API Planning Document — Travelly

Student: S. Kajarsan
Project: Travelly — AI-Assisted Private Hire, Rental, Route/Airport Ticket Hire and Travel-Social Platform
API style: REST/JSON over HTTPS | Base path: /api/v1 | Backend target: FastAPI
Database: PostgreSQL 16 + PostGIS | Schema: Travelly final 20-table NBP architecture
Date: 21 October 2026

Step 1 - Convert Use Cases into API Functions

For each Travelly use case, the frontend either requests information or submits a command to the backend. The table below converts the approved NBP use cases into API responsibilities. Database transactions are required for seat confirmation, vehicle availability, Driver acceptance, payment reconciliation and protected replacement.

Use Case	Required API Function	Primary Resource(s)
Register/login/logout	create account, issue/refresh/revoke tokens	auth, users
Manage shared profile	read/update current user profile	users
Request Owner or Driver role	submit role request; Admin approve/reject/suspend	roles, documents
Upload/verify documents	secure upload, list, review and expiry status	documents
Register vehicle	create/update/read Owner vehicle and media metadata	vehicles
Configure offered services	enable service and Owner pricing/terms for vehicle	service-types, vehicle-services
Search private hire/rental	filter eligible vehicles by service, time, capacity, luggage and availability	vehicles, vehicle-services, bookings, vehicle-records
Manage routes/stops	list/search routes; Admin create/update ordered stops	routes, route-stops
Publish Ticket departure	create draft, validate, publish and manage departure	scheduled-trips
Manage 2D seat layout	create/update versioned vehicle seat map	seat-layouts
View/hold Ticket seat	read departure seats; atomically create/release expiring hold	scheduled-trip-seats, seat-holds
Create booking	create private hire, Ticket or basic rental booking	bookings, travellers

Owner decision	from the Owner dashboard, reject the hire, accept as an eligible self-driver, or send a pending assignment to an eligible external Driver	bookings, trip-assignments
Driver decision	external assigned Driver views and accepts/declines the pending assignment; Owner self-drive needs no second acceptance	trip-assignments, trips
Travelly Protected fulfilment	commit original trip or propose/approve eligible replacement	scheduled-trips, replacements, bookings
Operate trip	ready/start/location/breakdown/complete	trips
Pay/refund/reallocate/payout	create payment intent, process webhook, refund/reallocate and view payable state	payments
Maintain vehicle	add/query maintenance, fuel, charging, odometer, personal use and blocks	vehicle-records
Notify/reconfirm	list/read notifications and record current in-app/email delivery/action response; IVR remains pending and unimplemented	notifications
Use Travelly Reels	feed, create/read/update/delete post and interactions	social-posts, social-interactions
Use road alerts	create, query nearby, confirm/deny and moderate expiring alerts	road-alerts
AI search/suggestion	extract validated filters and query actual inventory	ai, routes, departures, vehicles
Admin operations	review roles/docs/vehicles, manage catalogue, fulfilment and moderation	admin plus domain resources

Step 2 - Identify API Resources

Resources are the main objects and workflow boundaries exposed by the API. They are based on the final database design but are not required to have a one-to-one relationship with tables.

Identity and verification resources

- auth
- users
- roles
- documents

Fleet and service resources

- vehicles
- service-types
- vehicle-services
- vehicle-records

Route, departure and inventory resources

- routes
- route-stops
- scheduled-trips
- seat-layouts
- scheduled-trip-seats
- seat-holds

Commercial and operational resources

- bookings
- travellers
- trip-assignments
- trips
- payments
- refunds
- replacements
- notifications

Community, safety and AI resources

- social-posts
- social-interactions
- road-alerts
- ai/travel-search
- ai/trip-suggestions
- ai/road-alert-classification

Role-specific dashboards are response views assembled from these resources, not separate database entities. One shared account may switch among its approved Passenger, Owner and Driver contexts; pending roles cannot access their protected dashboards.

Step 3 - Select the Correct HTTP Method

Choose the method according to the operation. GET must not change state. POST creates a record or executes a non-idempotent business action. PATCH changes selected fields or performs an explicit state transition. DELETE is used only for deletable drafts, temporary holds or soft-deletable user content.

Method	Purpose	Travelly Example
GET	Read or search data	GET /api/v1/scheduled-trips?origin=Jaffna&destination=Colombo
POST	Create data or execute a business action	POST /api/v1/bookings/ticket-holds
PUT	Replace a complete editable resource	PUT /api/v1/vehicles/{vehicle_id}/seat-layout
PATCH	Partially update or perform a controlled transition	PATCH /api/v1/trips/{trip_id}/start
DELETE	Remove a draft/temporary/soft-deletable resource	DELETE /api/v1/bookings/{booking_id}/seat-hold

Action endpoints such as /publish, /accept, /start, /complete and /cancel are used when the operation applies business rules beyond ordinary field updates.

Step 4 - Plan Status Codes

All endpoints return the appropriate HTTP status plus a stable application error code. Validation errors must identify the affected field. Internal stack traces and sensitive provider errors must never be returned to clients.

Status	Meaning	Typical Travelly Use
200 OK	Request completed successfully	GET result or PATCH action with response body
201 Created	New resource created	registration, vehicle, booking, post or alert
202 Accepted	Asynchronous work accepted	media processing or queued notification
204 No Content	Successful action without response body	release hold or delete draft
400 Bad Request	Malformed request or invalid parameter combination	invalid date range or unsupported filter
401 Unauthorized	Missing, expired or invalid access token	protected endpoint
403 Forbidden	Authenticated but role/ownership/assignment disallows action	Passenger attempts Owner/Admin action
404 Not Found	Resource not found or not visible to caller	unknown booking/vehicle

409 Conflict	Current state or inventory conflicts	seat already booked, overlapping vehicle or stale transition
410 Gone	Temporary resource expired	seat hold expired
422 Unprocessable Entity	Structured validation failed	Pydantic field/cross-field validation
429 Too Many Requests	Rate limit exceeded	login, AI, upload or alert abuse protection
500 Internal Server Error	Unexpected backend failure	safe generic error and trace ID
502 Bad Gateway	External provider returned invalid/failing response	payment/AI/storage provider error
503 Service Unavailable	Temporary dependency or service outage	provider/database unavailable

Step 5 - API Plan

Complete Travelly API Plan (Google Sheet)

The detailed API plan is maintained in the linked spreadsheet using the lecturer's 12-column format. It contains 96 Travelly endpoints with request/input, validation rules, possible errors, success and error responses, authentication requirements, and related database tables.

S.kajarsan Travelly API plan

Step 6 - Plan the AI Endpoint

The frontend calls Travelly FastAPI, never the AI provider directly. The backend sends only the minimum required input, validates the structured output and then queries real Travelly inventory.
React / Mobile Web → FastAPI → Input validation and optional speech-to-text → AI extraction → Backend inventory filters → FastAPI response → React / Mobile Web

Primary AI endpoint planning sheet

Item	Student Entry
Endpoint	POST /api/v1/ai/travel-search

Method	POST
Who can use it?	Public user or authenticated Passenger; rate-limited
User input	input_type (text/voice transcript), query, optional locale and confirmed fields
Database context used	active routes/stops, service types, approved vehicles/services, published departures, seat availability, booking/trip/availability conflicts
Prompt purpose	extract structured travel intent; never invent a vehicle, fare, departure or preference
Expected AI output	intent, origin, destination, date/time, passengers, luggage, explicit preferences, confidence and clarification fields
Backend result	validated filters plus actual eligible private-hire/rental vehicles and Ticket departures
Will response be stored?	Raw prompt/AI response: No. Only an ordinary booking is stored if the user proceeds.
Failure message	“We could not confidently understand some trip details. Please confirm the highlighted fields or use the standard search form.”

Additional controlled AI endpoints

POST /api/v1/ai/trip-suggestions — converts an open request such as “I have three days; where can I travel?” into a suggested itinerary and then queries actual eligible routes/vehicles. Suggestions are clearly labelled and are not bookings.
POST /api/v1/ai/road-alerts/classify — converts a short text/transcript into a supported road-alert type and confidence score. The alert is saved only after validation and an active-trip/location check.
Conversational AI calling is excluded. Telephony/IVR remains pending and is not implemented in the current NBP. If later required and separately approved, authenticated Press 1/Press 2 outcomes may reuse the notification response contract.

Step 7 - Plan Pydantic Schemas / DTOs

Request and response models are separate. Password hashes, internal provider secrets, moderation notes, private document storage keys and fields belonging to other users are never exposed in public responses.

Schema / DTO	**Purpose**	**Typical Fields**
RegisterRequest	Create account	full_name, email or phone, password, preferred_language
LoginRequest / TokenResponse	Authenticate	identifier, password / access_token, refresh_token, expires_in

UserResponse / UserUpdate	Safe profile read/update	id, display_name, profile_image_url, contacts, language, approved_roles
RoleRequestCreate / RoleReview	Apply/review role	role_code, document_ids / decision, reason, version
DocumentCreate / DocumentResponse	Upload metadata/read safe result	owner_type, owner_id, document_type, number, dates, upload token / status
VehicleCreate / VehicleUpdate	Owner vehicle data	registration, type, make, model, year, power/fuel, seats, luggage, odometer, media
VehiclePublicResponse	Passenger-safe vehicle view	id, public reference, type, capacity, luggage, media, Owner summary, enabled services
VehicleServiceConfig	Owner service/pricing	service_type_id, enabled, pricing_basis, prices, currency, terms
RouteResponse / RouteAdminUpsert	Route and ordered stops	code, name, origin, destination, distance, duration, stops
SeatLayoutUpsert	Create versioned 2D layout	layout_version, seats[{seat_code,row,column,deck,type,bookable}]
ScheduledTripCreate	Departure draft	route_id, vehicle_id, departure_at, arrival_at, fare, policy, minimum_seats, accepted limits
ScheduledTripResponse	Public/Owner departure view	reference, route/stops, current vehicle, times, fare, assurance, seat summary, status
SeatMapResponse	2D inventory	seat_id, seat_code, row, column, type, status, hold_expiry, fare
TicketHoldRequest	Atomically hold seats	scheduled_trip_id, traveller drafts, seat_ids, boarding/drop stops, idempotency_key
PrivateHireCreate	Private hire request	service_type_id, origin, destination, start/end, passengers, luggage, vehicle preference
RentalCreate	Basic rental request	vehicle_id, start/end, licence_document_id, itinerary/terms acceptance
BookingResponse	Unified booking view	reference, type, itinerary, assurance, original/current fulfilment, totals, payment/trip summaries, status
CancellationRequest	Cancel booking	reason_code, explanation, version
OwnerDecisionRequest	Accept/decline/choose operator	decision, self_drive or driver_user_id, reason, version
DriverAssignmentResponse	Driver work item	trip_id, booking summary, vehicle, schedule, assignment_status
ReplacementProposalRequest	Protected fulfilment proposal	replacement_vehicle_id or scheduled_trip_id, time/route/seat/luggage comparison, reason

AlternativeResponseRequest	Passenger material-change decision	accept, proposal_reference, version
TripStartRequest	Start operation	start_odometer_km, current_location, checklist_confirmed, version
TripLocationUpdate	Current active location	latitude, longitude, accuracy_m, speed_kph, recorded_at
TripCompleteRequest	Finish operation	end_odometer_km, completed_at, notes, version
PaymentIntentRequest / Response	Create payment	booking_id, amount, currency, method, idempotency_key / provider client data
RefundRequest	Create related refund	amount or full, reason, idempotency_key
VehicleRecordCreate	Maintenance/expense/block	record_type, recorded_at, odometer, time range, amount, energy, next_due_km/date, notes
MaintenanceSummaryResponse	Calculated reminder view	current_km, next_due_km, remaining_km, average_daily_km, estimated_date, calendar_date, due_state
NotificationResponse	User notification	id, type, channel, title, body, action, status, timestamps
SocialPostCreate / Response	Create/read Reel/photo/review	post_type, media token, caption, hashtags, location, linked resource / moderation and counts
SocialInteractionCreate	Like/comment/save/share/report/rate	interaction_type, comment_text, rating, report_reason
RoadAlertCreate / Response	Create/read route alert	trip_id, alert_type, coordinates, direction, source_input, description / status, expiry, confirmations
AITravelSearchRequest	AI input	input_type, query or transcript, locale, confirmed_fields
AITravelSearchResponse	Structured filters plus real results	intent, extracted_fields, confidence, clarification_fields, vehicles, departures
ErrorResponse	Stable client-safe error	code, message, field_errors, trace_id
PaginatedResponse[T]	Consistent list response	items, page, page_size, total, has_next

Final API Security and Integrity Rules

1. Authentication proves identity; role approval, ownership and trip assignment determine permission.
2. Seat hold and confirmation execute in a database transaction with row locking or an atomic conditional update.
3. Vehicle/departure availability is checked again at confirmation using the overlap rule.
4. Payment, booking, scheduled-trip and operating-trip status transitions remain separate.

5. Travelly Protected replacement preserves accepted route, stop, time, capacity, luggage and quality limits; material changes require Passenger acceptance.
6. Owner self-drive requires an approved Driver role, valid documents and no schedule conflict.
7. No-response to a reconfirmation request does not automatically cancel or release inventory. The same rule applies if the pending optional IVR channel is enabled later.
8. Location updates are accepted only from the assigned active operator and stop after trip completion.
9. AI output never creates inventory, price or confirmation without ordinary backend validation.
10. Rate limits, audit-safe logs, CORS, input limits, signed object-storage uploads and webhook signature checks are required.
11. Online Ticket seats require full online payment before confirmation. NBP cash is limited to enabled private-hire/basic-rental services with Owner-selected start/end collection timing and two-sided Passenger plus collector confirmation.
12. Only the assigned active Driver or eligible self-driving Owner may create road alerts; Passengers may receive and submit feedback.
13. Admin configuration is bounded to service-specific settings and per-vehicle thresholds in the approved 20-table schema; a global rules engine is Phase 2.

Final API Completion Checklist

☑ Every approved NBP use case has an endpoint or documented server-side process.
☑ Every endpoint maps to the final 20-table database design.
☑ Passenger, Owner, Driver and Admin permissions are defined.
☑ 2D seat hold, booking conflict and vehicle overlap rules are covered.
☑ Fixed Departure and Travelly Protected fulfilment are covered.
☑ Ticket full-online-payment, restricted cash verification, idempotency, refunds, reallocations and Owner payable flow are covered.
☑ Maintenance, notifications, Reels, role-restricted road alerts, object-storage media and AI search are covered.
☑ Subscription and conversational AI calling are excluded; Telephony/IVR remains pending and unimplemented unless later required and approved; full chat and other Phase 2 capabilities are not introduced accidentally.
☑ This plan is ready for OpenAPI implementation and backend route development.